

Smart Contracts, Tokens y Aplicaciones

Ethereum como plataforma programable: reglas, activos digitales y coordinación económica sin intermediación central.

Mapa de la clase

Del contrato autónomo al ecosistema de aplicaciones

Contrato

reglas + estado

Token

propiedad digital

Dapp

experiencia de usuario

Ecosistema

composabilidad

Riesgo

seguridad y gobernanza

La pregunta no es sólo qué código ejecuta Ethereum, sino qué relaciones económicas y jurídicas estamos codificando.

Qué es un smart contract

Un programa persistente con dirección, estado y reglas ejecutables

Idea esencial

- Vive en una dirección de Ethereum.
- Conserva estado entre transacciones.
- Ejecuta funciones cuando recibe llamadas.
- No “entiende” el mundo: aplica reglas formales.
- Sus efectos son públicos, verificables e irreversibles salvo lógica de compensación.



Anatomía interna

Las piezas mínimas que debe reconocer un lector de contratos

Estado

variables persistentes

balances, owner, reservas

Funciones

entradas → reglas → efectos

transfer(), mint(), swap()

Eventos

huellas legibles fuera de la cadena

Transfer, Approval, Deposit

Permisos

quién puede hacer qué

onlyOwner, roles, DAO

Errores

frenos de ejecución

require, revert, custom errors

Lectura docente: un contrato no es “un documento legal digital”. Es una máquina de estado que acepta transacciones y produce cambios verificables.

Del usuario al cambio de estado

La dapp oculta complejidad; Ethereum conserva la regla verificable

1 Frontend

elige acción

2 Wallet

firma

3 RPC / nodo

envía

4 Contrato

ejecuta

5 Estado + logs

actualiza

Clave didáctica

El frontend puede cambiar; la wallet puede cambiar; el RPC puede cambiar. Lo que importa para la confianza es que el contrato ejecutado y el estado resultante sean verificables por la red.

Solidity: modelo mental

No hace falta memorizar sintaxis; sí entender dónde vive cada dato

storage

persistente

costoso

balances, owners, reservas

memory

temporal

barato relativo

cálculos intermedios

calldata

entrada externa

sólo lectura

parámetros de llamada

Regla de oro: todo cambio persistente consume recursos y debe ser defendible económica y técnicamente.

Tokens: de reglas a activos

Los estándares convierten contratos en piezas interoperables

Un token no es un archivo:

- Es un contrato con una interfaz conocida.
- Representa balances, propiedad o derechos.
- Se integra con wallets, mercados y protocolos.
- Su valor depende de derechos, liquidez, riesgo y gobernanza.

ERC-20

fungible

monedas, stablecoins, governance

ERC-721

único

NFTs, identidad, arte, entradas

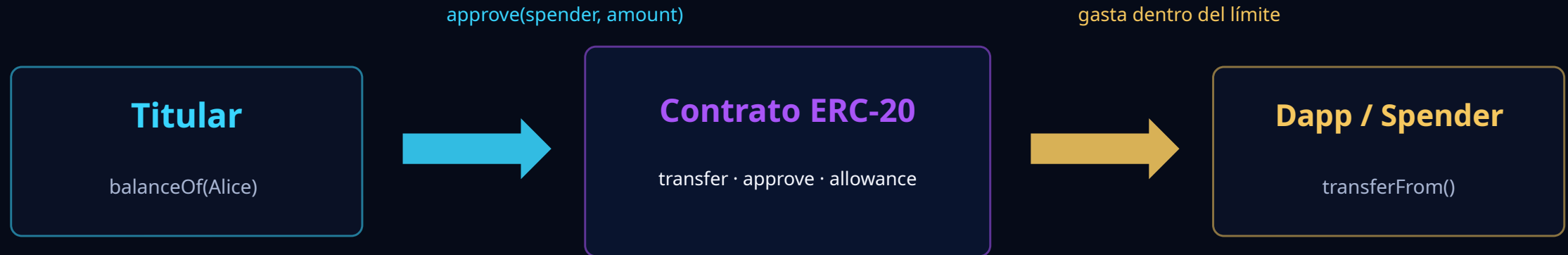
ERC-1155

multi-token

juegos, lotes, activos mixtos

ERC-20: dinero programable y autorización

El patrón approve/allowance permite delegar gasto, pero introduce riesgo



Riesgo típico

Aprobaciones ilimitadas o permisos olvidados pueden exponer fondos si el contrato autorizado falla, cambia o es explotado.

NFTs y multi-tokens

ERC-721 y ERC-1155 resuelven problemas distintos

ERC-721

un token = una identidad

propiedad única; metadata externa; marketplaces

ID único

tokenId indivisible

ERC-1155

un contrato = muchos tipos

lotes, juegos, inventarios, eficiencia

clases

balances por id

Advertencia docente: "NFT" no significa automáticamente derecho legal sobre una obra; hay que distinguir token, metadata, archivo, licencia y custodia.

Composabilidad: protocolos como legos

La innovación surge cuando contratos públicos se llaman entre sí

DEX

Stablecoin

Dapp nueva

Oracle

Lending

Wallet

Ventaja: velocidad de construcción. Riesgo: dependencia de piezas externas, liquidez, governance y errores heredados.

Oráculos: el puente con el mundo externo

La cadena no consulta internet por sí sola: alguien debe traer datos

Problema del oráculo

- Precios, clima, resultados, reservas o identidad viven fuera de la cadena.
- El contrato sólo puede ejecutar sobre datos que recibe.
- Si el dato es falso, la ejecución puede ser perfectamente incorrecta.
- La solución no es “creer”; es diseñar fuentes, incentivos y verificaciones.

Mundo real



Oracle network



Smart contract

Seguridad: pensar como atacante

En contratos, el error no sólo rompe una app: mueve valor

- 01  **Reentrancy** llamadas externas antes de actualizar estado
- 02  **Access control** funciones críticas sin permisos correctos
- 03  **Oracle manipulation** precios o inputs manipulables
- 04  **Upgrade risk** admin keys, proxies y cambios opacos
- 05  **Economic exploits** flash loans, slippage, liquidez débil

Práctica recomendada: minimizar privilegios, probar invariantes, auditar dependencias, usar bibliotecas maduras y diseñar mecanismos de emergencia claros.

Inmutabilidad, upgrades y gobernanza

El dilema: confianza por código estable vs necesidad de corregir

Contrato inmutable

predecible y verificable

difícil corregir errores

Proxy upgradeable

permite evolución

introduce poder administrativo

DAO / timelock

legítima cambios

más lento y coordinado

La gobernanza también es arquitectura: define quién puede cambiar las reglas que custodian valor.

Arquitectura de una dapp real

Más que un contrato: una pila completa de interacción y datos

Usuario

wallet, firma, UX

Frontend

web/app, librerías, estados

RPC / nodos

lectura y envío de transacciones

Contratos

reglas, estado, eventos

Indexadores

eventos → consultas rápidas

Storage externo

IPFS, metadata, archivos

Un buen diseño separa confianza crítica en cadena, experiencia de usuario fuera de cadena y datos indexados para consulta eficiente.

Casos de uso: qué se programa

El contrato codifica derechos, flujos, restricciones y coordinación

DeFi

swaps, préstamos, vaults, stablecoins

Tokenización

activos, puntos, participaciones, certificados

NFTs / gaming

propiedad digital, inventarios, royalties

DAOs

votos, tesorería, propuestas, ejecución

Trazabilidad

eventos verificables, anclajes, evidencia

Qué se programa: derechos, flujos, restricciones e incentivos.

Laboratorio sugerido

De la teoría a una práctica evaluable

- 1 Deploy** contrato ERC-20 simple en testnet/local
- 2 Interactuar** transfer, approve y lectura de balances
- 3 Eventos** leer logs y reconstruir actividad
- 4 Riesgo** revisar permisos, owner y approvals
- 5 Informe** supuestos, gas, UX y seguridad

Producto final: demo funcionando + tabla de transacciones + lectura de eventos + mini auditoría de riesgos.

Cierre conceptual

Ethereum como infraestructura de coordinación

Contratos

formalizan reglas y estado

Tokens

estandarizan propiedad e intercambio

Oráculos

abren la puerta al mundo externo

Dapps

traducen complejidad en experiencia

Seguridad

define si el sistema merece confianza

La promesa de una aplicación descentralizada no está en el marketing, sino en sus invariantes: qué puede pasar, quién puede cambiarlo y qué riesgo asume cada actor.

Fuentes y continuidad

Base conceptual para profundizar y preparar el laboratorio

- [ethereum.org](#) — Introduction to smart contracts
- [ethereum.org](#) — ERC-20 Token Standard
- [ethereum.org](#) — ERC-721 Non-Fungible Token Standard
- [ethereum.org](#) — ERC-1155 Multi-Token Standard
- [ethereum.org](#) — Oracles
- [ethereum.org](#) — Ethereum Virtual Machine (EVM)

Próxima presentación: Escalabilidad, L2, rollups, blobs y disponibilidad de datos.